

Revision Control and GDB

CSC 230 : C and Software Tools

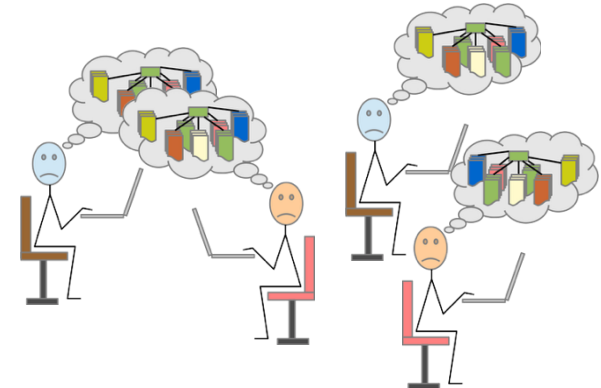
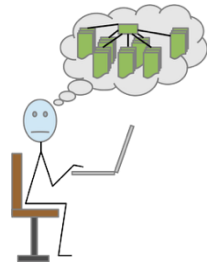
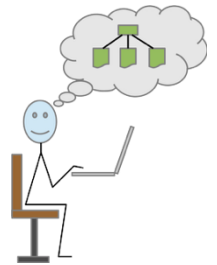
NC State Department of Computer
Science

Topics for Today

- Revision Control
- Meet Git
- Meet the Gnu Debugger
- More Git and GitHub

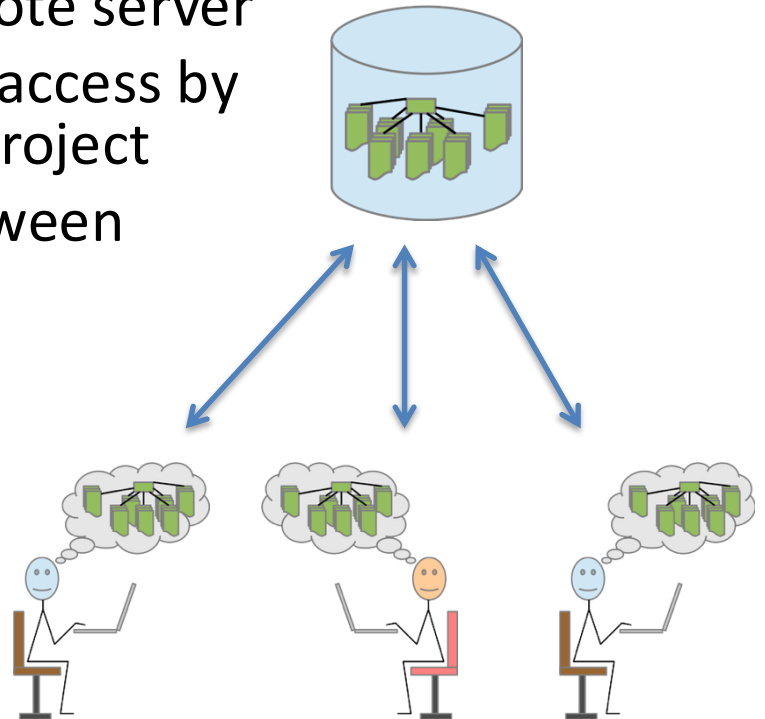
Why We Need Revision Control

- How do you store your source code for a project?
 - Maybe just as a tree of files on your computer.
- That's fine, as long as you never need to ...
 - Make changes to your source
 - Try out alternative versions of the same files
 - Backup your work
 - Coordinate your work with other developers

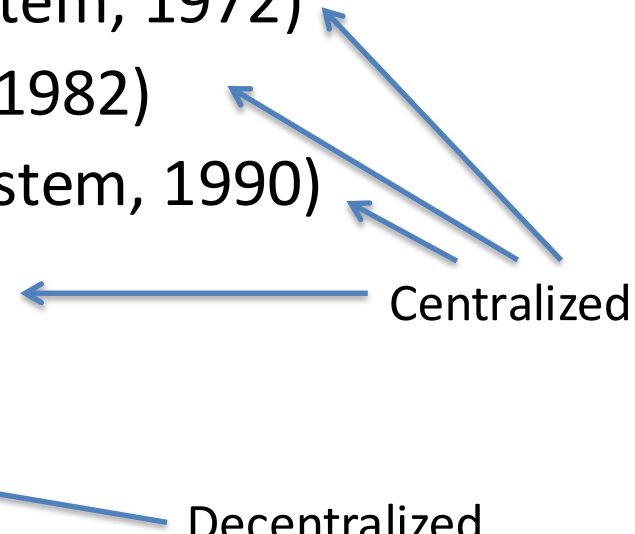


Revision Control Systems

- A Revision Control System helps with these problems
 - It maintains **multiple versions** of the project, including a version history
 - It can provide **backup** to a remote server
 - It provides **convenient, secure** access by many developers to a shared project
 - It provides **conflict control** between developers

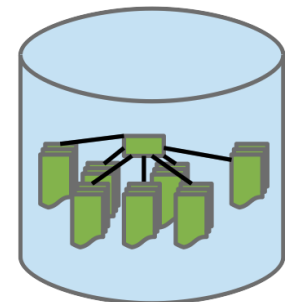
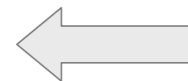
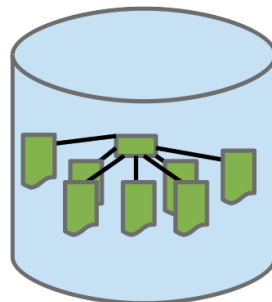
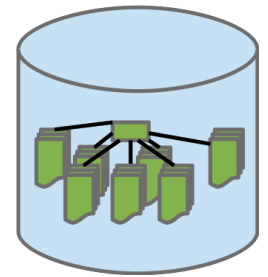


Revision Control ... Revisions

- We've developed many revision control systems
 - I've used all these (and some others)
 - SCCS (Source Code Control System, 1972)
 - RCS (Revision Control System, 1982)
 - CVS (Concurrent Versioning System, 1990)
 - Subversion (Subversion, 2000)
 - git and GitHub (2005)
- 
- The diagram consists of blue arrows pointing from the labels 'Centralized' and 'Decentralized' to specific revision control systems. The 'Centralized' label has three arrows pointing to SCCS, RCS, and CVS. The 'Decentralized' label has one arrow pointing to git and GitHub.
- Centralized
- Decentralized

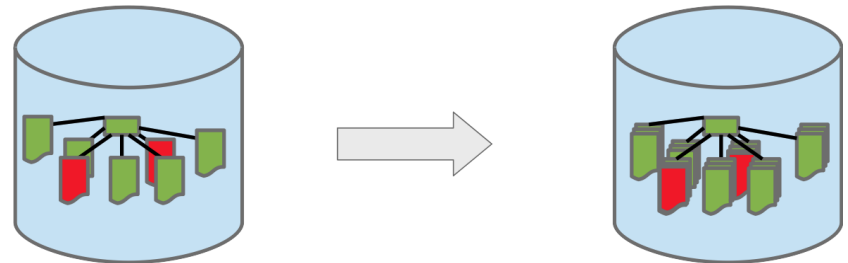
Revision Control Basics

- All revision control systems have some things in common.
- They define a *repository*, where master copies of project files are maintained
 - Along with a revision history
- You can *checkout* a working tree from the repository
 - Typically, the latest version



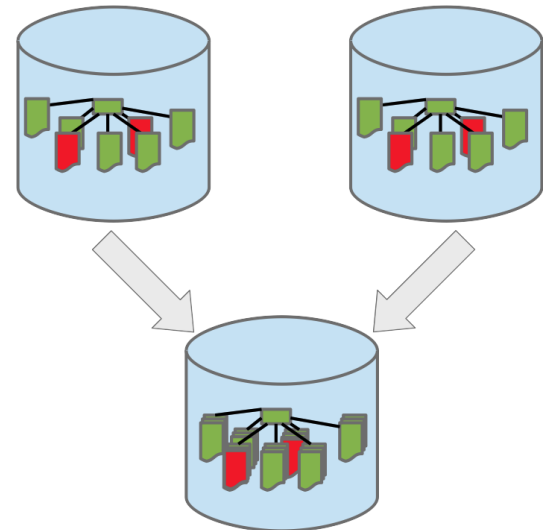
Committing a Version

- Source code changes
 - An update to the source may require multiple changes.
- There's a mechanism to *commit* related changes all at once.
 - Collect related changes, probably across multiple files.
 - Put them into the repository all at once.
 - That's what creates a *version* in the repository.



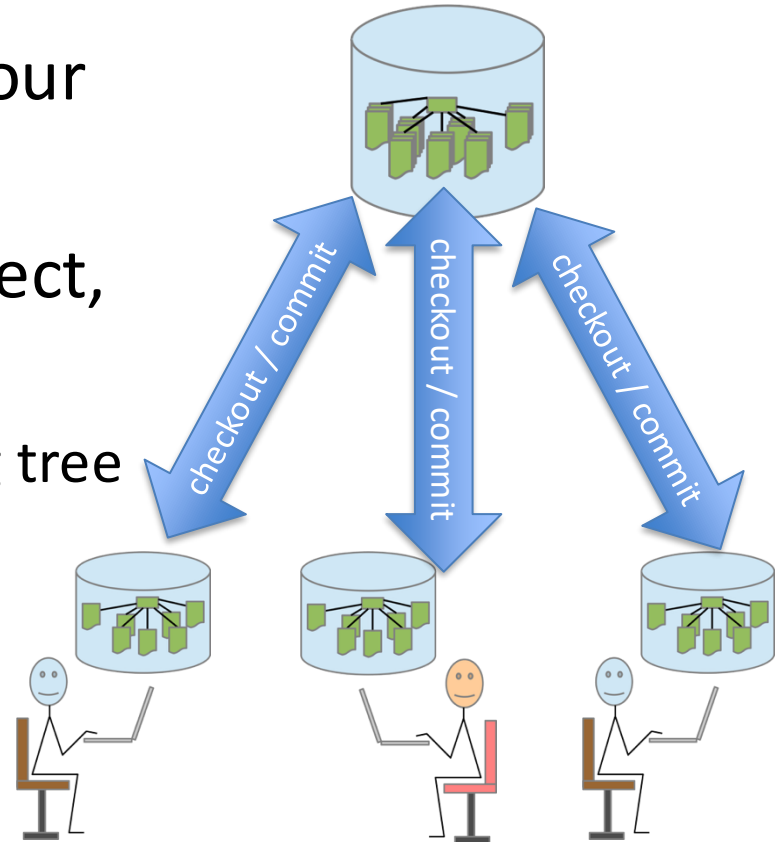
Coordinating Changes

- There's some mechanism to coordinate concurrent edits
 - Some systems let you lock files in the repository, while you're making changes.
 - It's more common now to let developers edit files concurrently.
 - Understanding that they may have to merge their changes later.



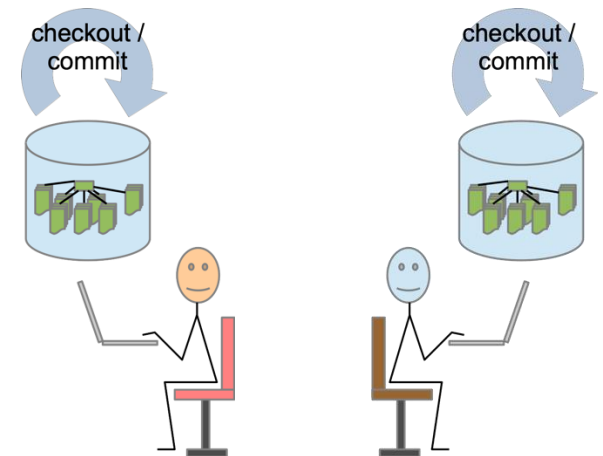
Centralized Revision Control

- Many revision control systems are centralized
 - There's a server where your project's repository lives
 - To work on a shared project, developers:
 - *Check out* a local, working tree for the project
 - *Commit* changes to the repository



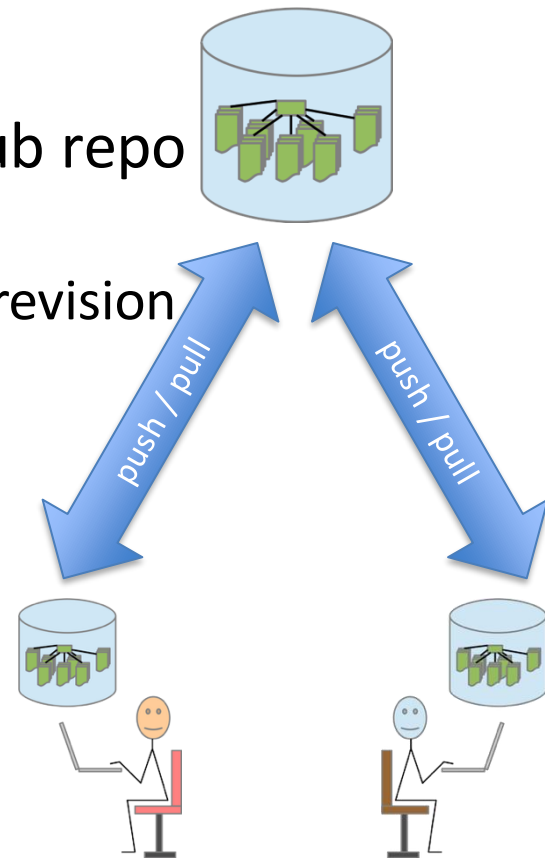
Git and Decentralization

- Git is a decentralized revision control system
 - Developed by Linus Torvalds for the Linux kernel source
- Every developer can have their own repository
 - Same idea, make a set of related changes, commit them
 - No central repository needed, you can commit locally
 - Every user's repository maintains a revision history
- How do developers cooperate?



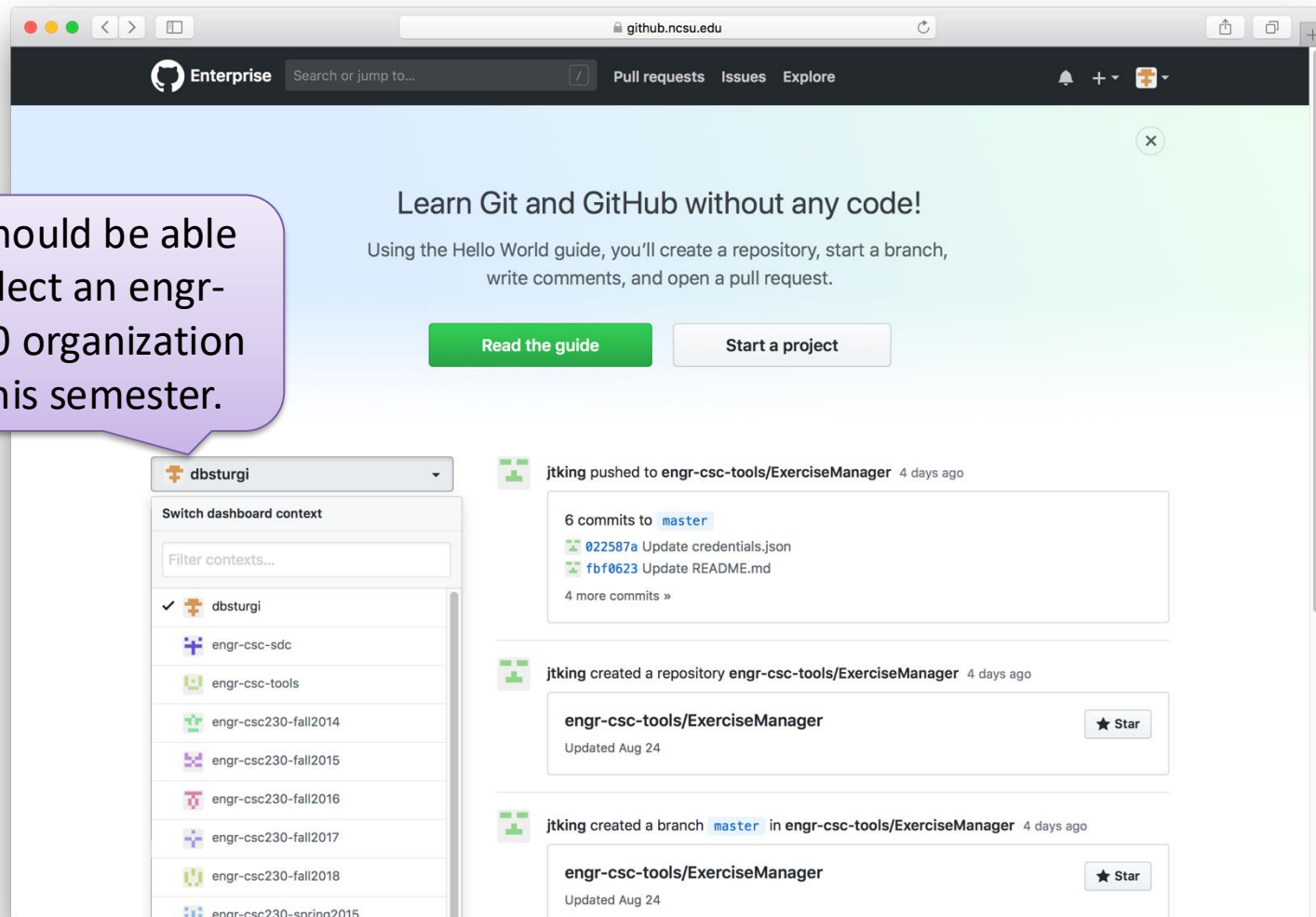
GitHub

- GitHub, a service that hosts Git repositories
 - Along with a UI for managing your repositories
 - There's github.com, available to everyone.
 - And, at NCSU we have github.ncsu.edu
- We can *push* revisions up to a remote github repo
- .. and *pull* revisions down to our local rep
 - So, we can end up using it a lot a centralized revision control system.
- Starting from Homework 2 we will use github.ncsu.edu for assignment submission
- Let's give it a try

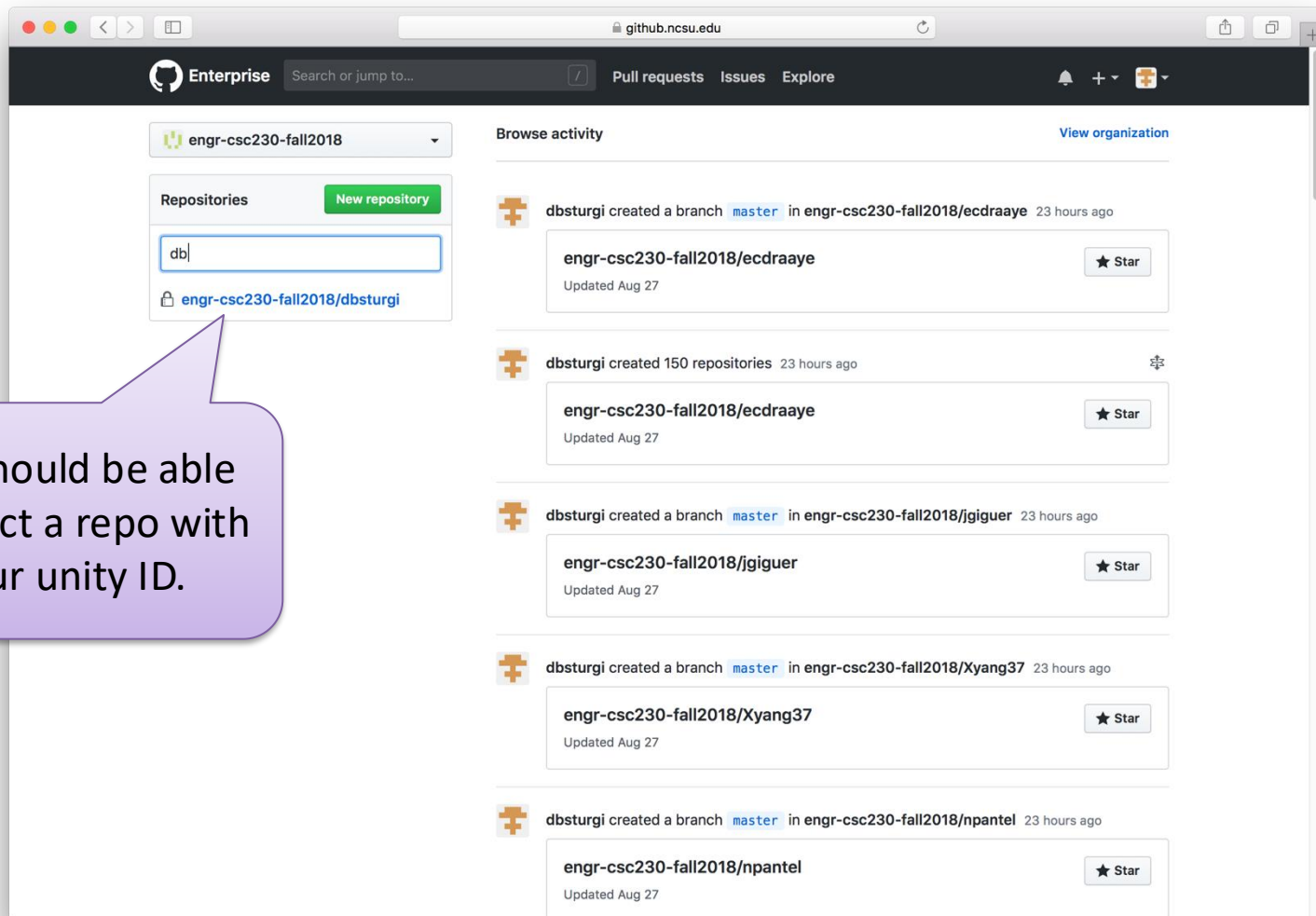


Visit github.ncsu.edu

You should be able to select an engr-csc230 organization for this semester.



Check out your New Repo



Get Ready to Clone

Enterprise Search or jump to... Pull requests Issues Explore

engr-csc230-fall2018 / dbsturgi Private

Unwatch 1 Star 0 Fork 0

<> Code Pull requests 0 Projects 0 Insights Settings

No description, website, or topics provided. Edit

Add topics

2 commits 1 branch 0 releases 1 contributor

Branch: master New pull request

Create new file Upload files Find file Clone or download

dbsturgi Initial repo structure

example	Initial repo structure
p2	Initial repo structure
p3	Initial repo structure
p4	Initial repo structure
p5	Initial repo structure
p6	Initial repo structure
.gitignore	Initial repo structure
README.md	Initial commit

README.md

dbsturgi

Clone with HTTPS Use SSH

Use Git or checkout with SVN using the web URL.

https://github.ncsu.edu/engr-csc230-f18/dbsturgi

Open in Desktop Download ZIP

From the repo, you should be able to copy a URL you can clone.

Using Your New Repo

- You can clone your repo from shell on a common platform system.

The URL
you just
copied.

```
$ git clone https://github.ncsu.edu/engr-csc230-spring2019/dbsturgi.git
Cloning into 'dbsturgi'...
Username for 'https://github.ncsu.edu': dbsturgi
Password for 'https://dbsturgi@github.ncsu.edu':
remote: Counting objects: 3, done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), done.
```

These will be
your unity ID.

Using Your New Repo

- Or, including your unity ID in your URL can save you some time later on.

The URL you
got from
github.

With your unity
ID added in.

```
$ git clone https://dbsturgi@github.ncsu.edu/.../dbsturgi.git
Cloning into 'dbsturgi'...
Password for 'https://dbsturgi@github.ncsu.edu':
remote: Counting objects: 3, done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-r
Unpacking objects: 100% (3/3), done.
```

See, you didn't
have to give your
unity ID here.

Using Your Repo

- The repo contains an example directory, with some of the example files for today.

```
$ cd dbsturgi
$ cd example
$ ls
garbage-file.txt  primes1.c  primes2.c  primes3.c
README.md
```

Whatever your
unity ID is.

- Let's try out the primes1.c program.

```
$ gcc -Wall -std=c99 -g primes1.c -o primes1
$ ./primes1
Floating point exception
```

Ouch.
We should debug
this program.

Meet GDB

- GDB : the GNU debugger
 - Debugger for C, C++ and other programs compiled with the GNU compiler suite
- GDB will let you
 - Run and resume your program's execution
 - Set breakpoints at particular source lines
 - Examine the program state
 - Change things in the program.

GDB and the IDE

- gdb is a *symbolic debugger*.
 - It lets us work with the names in our source code ... instead of just memory addresses.
- Like the compiler, it's the kind of tool that's often hidden inside a user-friendly IDE
 - e.g., IDEs like Eclipse can provide a GUI for gdb
- It's a tool we can use directly if we're working from the command line

Preparing for GDB

- To get the most out of gdb, you need to use the -g flag when you compile.
 - This tells the compiler to include symbol names and extra information not strictly necessary for running your program.
 - But, gdb can use this information to show you more of what's going on inside your program.
- So, we'll want to compile like this:

```
$ gcc -Wall -std=c99 -g myProgram.c -o myProgram
```

GDB Demonstration

- We'll try out gdb using a small collection of bad programs:
 - primes1.c : is supposed to report prime numbers ... but it crashes.
 - primes2.c : is supposed to report prime numbers ... but it gets stuck in an infinite loop.
 - primes3.c : is supposed to report prime numbers ... but it only gets a couple of them.
- The next few slides show what we're supposed to learn from working with gdb

Running with GDB

- To debug our program, we'll need to start gdb and tell it to work with our program:

```
$ gdb ./myProgram
```

- You can also attach gdb to a program that's already running (handy for debugging a program you didn't expect to debug)

```
$ gdb ./myProgram processID
```

- Then, you can set breakpoints by function name.

```
(gdb) break someFunctionName
```

- Or by line number:

```
(gdb) break 25
```

Running with GDB

- When you're ready, you can start your program running:

```
(gdb) run
```

- If you need to do I/O redirection, this is where you do it.

```
(gdb) run < input-file.txt > output-file.txt
```

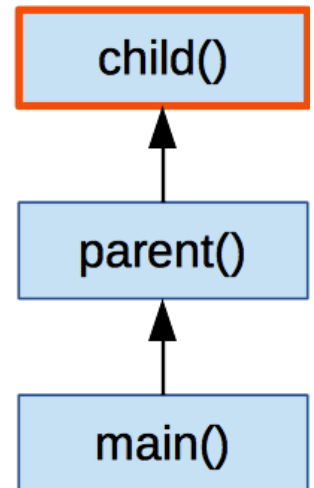
- **Not** like this:



```
$ gdb ./myProgram < input-file.txt > output-file.txt
```

Looking Around with GDB

- Then, gdb will run until our program
 - terminates
 - or until it crashes
 - or until it hits a breakpoint.
- Then, you'll get the gdb prompt back.
- You can look at a *stack trace*, a list of the function calls that got the program to this point:



```
(gdb) backtrace
#0 child (x=3,y=99)
#1 parent (param=7) at ...
#2 main () at ...
```


Looking Around with GDB

- You can look at the values of local variables:

```
(gdb) print i
```

- Or, really, any expression:

```
(gdb) print i * n + j
```

- You can even change the value of a variable:

```
(gdb) set variable n = 25
```

Looking Around with GDB

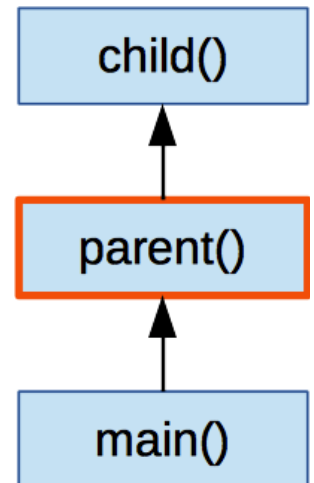
- You can move up a stack frame
 - To look at variable values in the caller

```
(gdb) up
```

- This doesn't end the current function ...
you're just looking around

- You can move both directions in the stack trace

```
(gdb) down
```



Looking Around with GDB

- GDB only shows a little bit of source code at a time.
- You can look at surrounding source lines:

```
(gdb) list
```

- Or, code for any function you want.

```
(gdb) list someFunctionName
```

- Or, if you run gdb with the -tui option, it will try to show your source code in part of the terminal window.

```
$ gdb -tui ./myProgram
```

ctrl-L is convenient for re-painting the UI.

Resuming Execution

- You can resume running your program
 - Until it terminates, hits a breakpoint, etc.

```
(gdb) continue
```

- You can run to just the next instruction (stepping over any function calls)

```
(gdb) next
```

Resuming Execution

- You can step ahead, going into any function calls:

```
(gdb) step
```

- You can run ahead to any line of code you want:

```
(gdb) until 46
```

- Or, run until the current function returns:

```
(gdb) finish
```

Wrapping Up

- While you're looking at your program's state, it's frozen in place.
- You can kill it when you're done:

```
(gdb) kill
```

- And, maybe re-run it later if you want.
- If you have a makefile (later), you can even re-build from the gdb prompt.

```
(gdb) make
```

Wrapping Up

- When you're done, you can quit gdb to get back to the shell

```
(gdb) quit
```

GDB Command Summary

- While your program is stopped, you can issue commands to gdb.

Command	abbrev	
break <i>line</i>	b	Set a breakpoint at the given line
break <i>sourcefile:line</i>	b	Set a breakpoint in a particular file
break <i>functionName</i>	b	Set a breakpoint at a given function
break <i>line</i> if <i>condition</i>	b	Only break if condition evaluates to true
run	r	Start running the program
run arg1 args2 < file.txt	r	Run with arguments and redirection
continue	c	Continue running the program

GDB Command Summary

- While your program is stopped, you can issue commands to gdb.

Command	abbrev	
next	n	Step over function calls to next statement
step	s	Single-step to next statement, entering function calls.
until <i>line-number</i>		Run until the given line number
finish		Finish running the current function

GDB Commands

- While your program is stopped, you can examine and change its state.

Command	abbrev	
print <i>expr</i>	p	Evaluate and print the given expression
set variable <i>var</i> = <i>expr</i>		Set the given <i>var</i> to the the value of <i>expr</i>
list	l	Show surrounding source file lines
list functionName	l	Show code around the given function
list lineNumber	l	Show code around the given line number

More GDB Commands

Command	abbrev	
backtrace	bt	Show a stack trace
up		Move up one stack frame
down		Move down one stack frame
delete <i>n</i>	d	Delete breakpoint <i>n</i>
ctrl-C		Break the currently running program
condition <i>breakpoint condition</i>		Make a breakpoint conditional
kill	k	Kill the program being debugged
quit	q	Exit gdb

Checking the Repo Status

- The `git status` command can tell us what files we've changed.

```
$ git status
# On branch master
# Changes not staged for commit:
# ...
#       modified:   primes1.c
#       modified:   primes2.c
#       modified:   primes3.c
#
# Untracked files:
# ...
#
#       prime-list.txt
```

Tracked files we changed.

A new file we created.

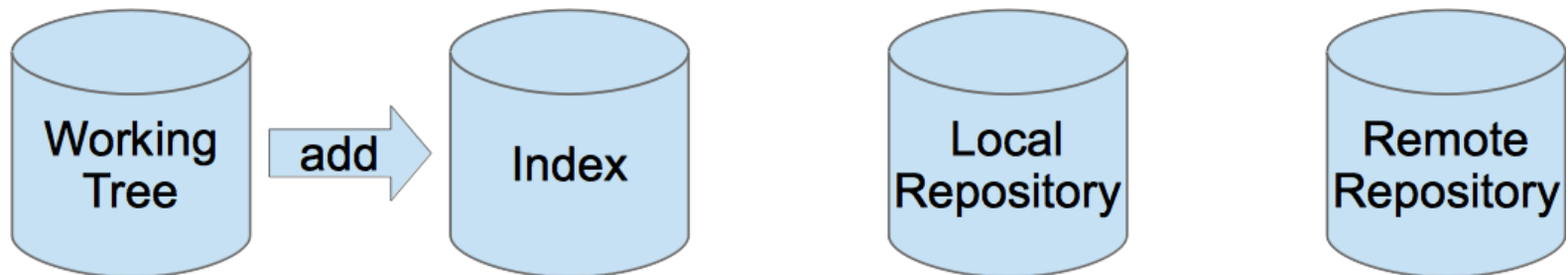
- But, we haven't changed anything in our repo yet.

Staging Files

- Tell git about the set of changes you want to commit.

```
$ git add primes*.c  
$ git add prime-list.txt
```

- Really, this is staging the current version of a file to the *index*
 - The record of files for the next commit.



Re-Checking Git Status

- See what's about to happen

```
$ git status
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
#    new file:   prime-list.txt
#    modified:  primes1.c
#    modified:  primes2.c
#    modified:  primes3.c
#
```



git status will give you advice about how to change things.

- It's a good idea to check status before committing any changes.

.ignoreance is Bliss

- You want to keep a clean repo
 - Without garbage or temporary files taking up space
 - There's a file size limit for the NCSU github!
- git helps with a file called **.gitignore**
 - It tells git files not to commit to the repo
 - Unix systems interpret this as a *hidden file*

```
# ignore object files  
*.o
```

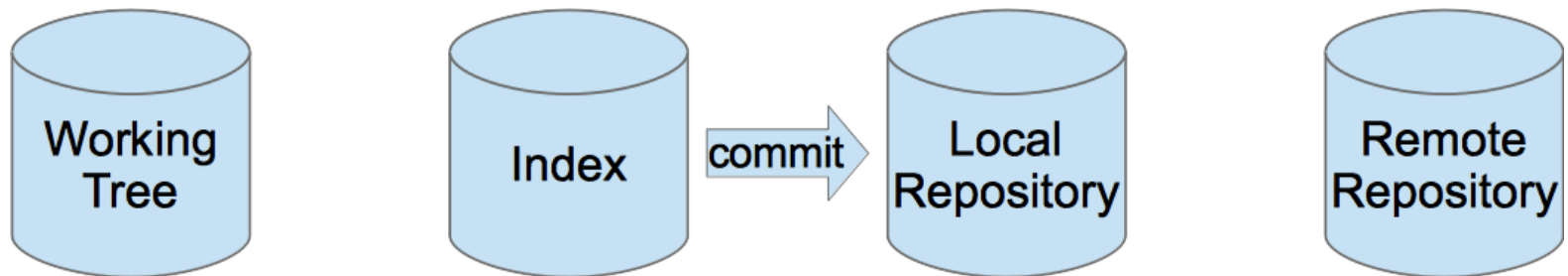
```
# ignore our temporary output files  
*/output.txt
```

Supports comments
and a pattern
matching syntax.

Working in Git

- When ready, commit to your local repo.

```
$ git commit
```



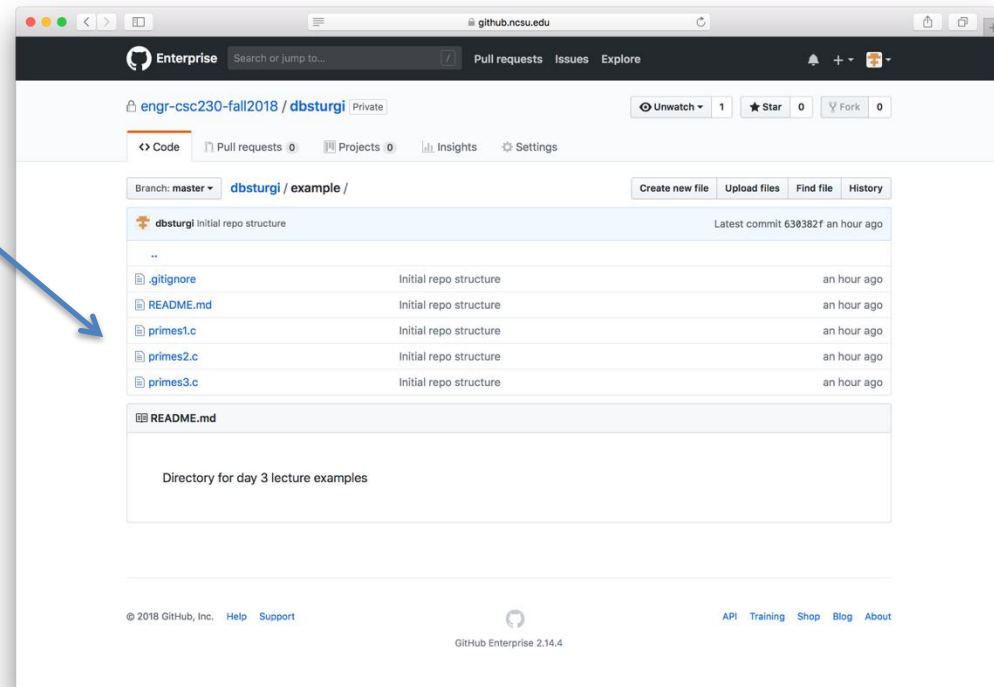
- Or, even better
 - -a : automatically add tracked files.
 - -m : here's a commit message

```
$ git commit -am "Fixed bugs in prime programs."
```


Working in Git

- But, the new file still isn't here.
- git status will tell you why.

```
$ git status
```

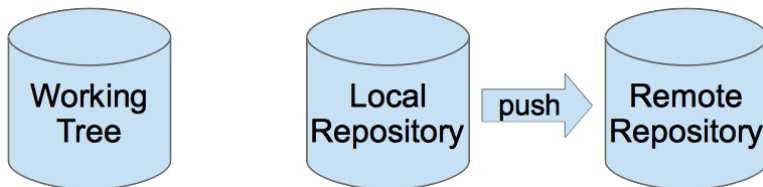
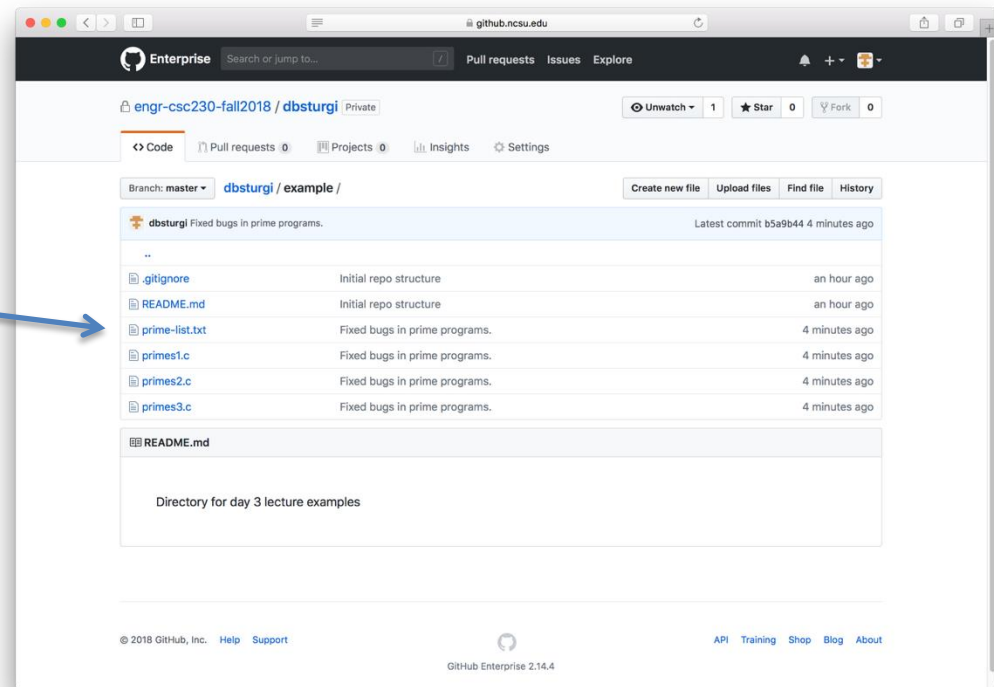


Push Your Changes

- Local commits must be pushed to a remote repo

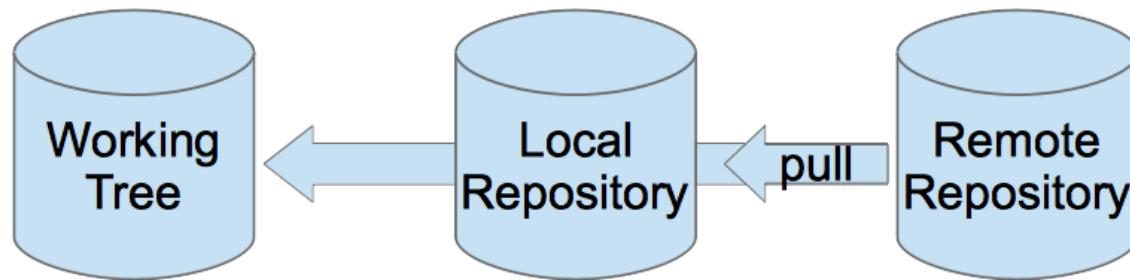
```
$ git push
```

- There it is.



Pull Updates

- Use pull to get a fresh copy of the repository



- So, you can share a remote repository
 - To share work between developers
 - Or, just to keep copies of your code in more than one place.

Pull Updates

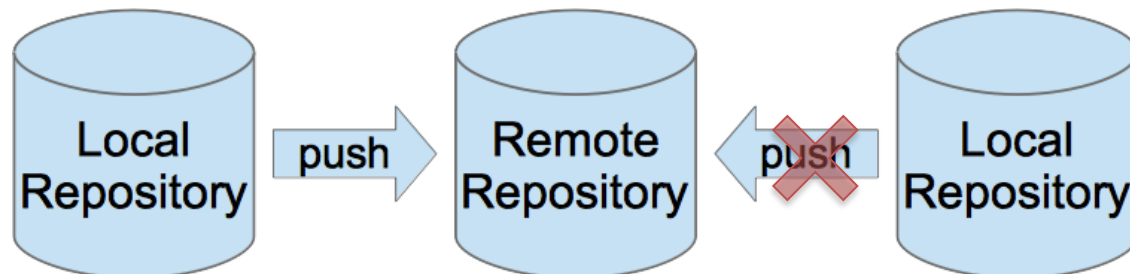
- There's a command for that:

```
$ git pull
remote: Counting objects: 4, done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 3), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (4/4), done.
From https://github.ncsu.edu/engr-csc230-fall2018/dbsturgi
   b5a9b44..a065f94  master      -> origin/master
Updating b5a9b44..a065f94
Fast-forward
 example/primes1.c | 2 --
 1 file changed, 2 deletions(-)
```

Conflicting Edits

- Conflicting edits may be committed to different copies of the repo.
 - Git doesn't want to clobber changes during a push
 - So. It will refuse to push if your local repo isn't completely up to date.

```
$ git push
To https://github.ncsu.edu/engr-csc230-fall2018/dbsturgi.git
! [rejected]        master -> master (non-fast-forward)
error: failed to push some refs to 'https://github.ncsu....'
```



Merging Edits

- Issuing a pull will try to merge the edits.

```
$ git pull --no-rebase
remote: Counting objects: 4, done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 3), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (4/4), done.
From https://github.ncsu.edu/engr-csc230-fall2018/dbsturgi
    04c3ba2..e618eb6  master      -> origin/master
Auto-merging example/primes1.c
CONFLICT (content): Merge conflict in example/primes1.c
Automatic merge failed; fix conflicts and then commit the result.
```

git will try to
merge text files if
it can.

... but it may need
help from you.

Merging Edits

- You may have to edit your source files ... to decide what parts of the code to keep.

```
...  
<<<<<<< HEAD  
#define LIMIT 10000  
=====  
#define LIMIT 5000  
>>>>>>> 1b27c3...  
...
```

Local version of
your code.

Version you just
pulled from the
remote repo.

- Edit the file, to make it look the way you want.

Merging Edits

- Now, you have a new version of the project.
- You'll need to:
 - `git commit -a`
You will get a default message for the merge.
 - `git push`

Git Workflow

- Clone your repository once (or for each system you plan to work on)

```
$ git clone url-for-my-repo
```

- Repeat

- Pull a fresh copy of the repository

```
$ git pull
```

If anyone else might be making changes.

- Edit, edit, edit
- Add changes to the index.

```
$ git add some-new-files-I-just-added
```

If you need to add new files to the repo.

- Commit the changes

```
$ git commit -am "a message about what changed"
```

- Push the changes

```
$ git push
```

Erasing Mistakes

- The style guide requires you to keep a clean repo
 - All the files you need ...
 - ... none of the files you don't
- What if you accidentally add a file that shouldn't be there?
- There's a git command for that: **git rm**

```
$ git rm some-file
```

- Removes the local file ... change goes into the repo with the next commit.

Erasing Big Mistakes

- You can remove multiple files at once:

```
$ git rm file1 file2 file3
```

- The -r option to rm will let you recursively remove whole directories.

```
$ git rm -r some-directory
```

Reflecting on the Past

- You can review recent commits with: **git log**

```
$ git log
commit fddc310e3d413832802e7269b90e75c455ac0670 (HEAD
Author: David Sturgill <dbsturgi@ncsu.edu>
Date:   Wed Jan 18 11:08:25 2023 -0500
```

Tried removing directory.

```
commit 44840a0295d61a552a9e32e44d171868f1695589
Author: David Sturgill <dbsturgi@ncsu.edu>
Date:   Wed Jan 18 11:06:46 2023 -0500
```

Deleted dummy files.

This is a commit /
version.

Versions are
identified by a
unique hash.

Revisiting the Past

- You can check out an older version with:
git checkout

```
$ git checkout 44840  
... message from git ...
```

Unique prefix of a hash.

- Now, you can look around an old state of your project.

Returning to the Present

- If you remember the hash, you can return to the present with `git checkout`:

```
$ git checkout fddc3  
... message from git ...
```

Hash from the most recent commit.

- Or, you can just name the branch you're on (main)

```
$ git checkout main  
... message from git ...
```

Using Git on Assignments

- Starting with Homework 2
 - You will use the NCSU github
 - We'll make a new repo for you.
 - A push to your repo is like a submission
- When you complete a project, you can check the web interface to make sure your changes made it.